

Day 126: Weekly Revision — HTTP, REST, Postman and API Testing

1. Day number

Day 126

2. Topic name

Weekly Revision — HTTP, REST, Postman and API Testing

Today you will revise **Days 121–125** and connect the main API concepts into one small mental model.

3. Connection

This revision combines the API foundations you need before building larger:

- backend applications
- FastAPI projects
- frontend/backend integrations
- AI applications that call external APIs
- ML model APIs

Your learning path this week was:

```
Day 121 → HTTP request/response
      ↓
Day 122 → REST + HTTP methods
      ↓
Day 123 → Parameters, headers, JSON, status codes
      ↓
Day 124 → Postman
      ↓
Day 125 → API testing + basic authentication
      ↓
Day 126 → Revision
```

The big picture is:

```
Client
  |
  | HTTP request
  ↓
API Server
  |
  | HTTP response
```

↓
Client

4. Revision summary of Days 121–125

Day 121 — HTTP Fundamentals

You learned the basic client/server relationship.

```
Client
  ↓ request
Server
  ↓ response
Client
```

For example:

```
GET /products/5
```

The client asks the server for product 5.

The server may respond with:

```
{
  "id": 5,
  "name": "Monitor",
  "price": 8000
}
```

Main ideas:

```
Client
Server
HTTP
URL
Request
Response
```

Day 122 — REST APIs and HTTP Methods

You learned that REST-style APIs commonly treat things such as products, customers, and orders as **resources**.

Example resource:

```
/products/5
```

You also learned the basic CRUD mapping:

```
Create → POST
Read   → GET
Update → PUT
Delete → DELETE
```

For products:

```
GET    /products/5 → read product 5
POST   /products  → create product
PUT    /products/5 → update product 5
DELETE /products/5 → delete product 5
```

Day 123 — Request and Response Details

You looked deeper inside HTTP messages.

A request may contain:

```
Method
URL
Path parameters
Query parameters
Headers
JSON body
```

A response may contain:

```
Status code
Headers
JSON body
```

You also learned common status codes:

```
200 → success
201 → created
400 → bad request
401 → authentication problem
404 → not found
500 → server problem
```

Day 124 — Postman Fundamentals

You learned that **Postman** can act as an API client.

Basic Postman workflow:

```
Choose method
↓
```

```
Enter URL
  ↓
Add parameters
  ↓
Add headers
  ↓
Add body if needed
  ↓
Send
  ↓
Inspect status code
  ↓
Inspect response
```

This lets you test APIs without first writing client-side Python code.

Day 125 — API Testing and Authentication

You learned not to test only successful requests.

You should also test failure situations.

For example:

```
Valid request
  ↓
Should succeed

Missing required input
  ↓
Should fail correctly

Missing/invalid authentication
  ↓
Should be rejected
```

You also learned:

```
Authentication
  ↓
Who are you?

Authorization
  ↓
What are you allowed to do?
```

And you learned that API keys should not normally be:

hard-coded into public source code
or
publicly shared

5. Important topics

Here are the major concepts from this week.

Topic	Simple meaning
HTTP	Rules for client/server communication
Request	Message sent from client to server
Response	Message returned by server
REST	Common way of organizing APIs around resources
GET	Read data
POST	Create data
PUT	Update data
DELETE	Delete data
Header	Extra information about an HTTP message
Path parameter	Value inside a URL path
Query parameter	Extra option/filter after ?
JSON	Common structured format for API data
Status code	Number describing the request result
Postman	Tool for manually sending API requests
API key	Credential used by some APIs

6. Foundational notes

A good beginner mental model is:

```
What do I want to do?  
  ↓  
Choose HTTP method  
  ↓  
Which resource?  
  ↓  
Build URL  
  ↓  
Do I need parameters?  
  ↓  
Do I need headers?
```

↓
Do I need a JSON body?
↓
Send request
↓
Server processes it
↓
Inspect status code
↓
Inspect JSON response

For example:

Goal:
Get product 7

Choose:

GET

Resource:

/products/7

Request:

GET /products/7

Possible successful response:

200 OK

```
{  
  "id": 7,  
  "name": "Keyboard",  
  "price": 1200  
}
```

7. Easy example

Imagine a tiny products API.

Base URL:

<https://api.example.com/products>

It contains these products conceptually:

```
1 → Laptop
2 → Keyboard
3 → Mouse
```

Reading one product

Request:

```
GET /products/2
```

Here:

```
GET
```

is the HTTP method.

```
/products/2
```

is the resource path.

And:

```
2
```

is a path parameter representing the product ID.

Possible response:

```
200 OK
```

```
{
  "id": 2,
  "name": "Keyboard",
  "price": 1200
}
```

Creating one product

Request:

```
POST /products
```

JSON body:

```
{
  "name": "Webcam",
}
```

```
"price": 2500
}
```

Possible response:

```
201 Created
```

```
{
  "id": 4,
  "name": "Webcam",
  "price": 2500
}
```

8. Revision problem statement

Design and conceptually test a very small products API.

You need only two operations.

Operation A — GET one product

Design an endpoint for retrieving product 10.

Include:

```
HTTP method
Path
One path parameter
Expected JSON response
Expected status code
```

Operation B — POST one product

Design an endpoint for creating this product:

```
{
  "name": "Laptop Stand",
  "price": 1500
}
```

Include:

```
HTTP method
Path
JSON request body
Expected JSON response
Expected status code
```

Also use **one path or query parameter** somewhere in your design.

For example:

```
/products/10
```

or:

```
/products?category=electronics
```

Finally, explain how you would test both requests manually using Postman.

Do **not** build a backend.

9. Concepts used

This revision problem uses:

```
HTTP
Client/server
Request
Response
REST
Resource
Endpoint
GET
POST
Path parameter
Query parameter
Headers
JSON
Status code
Postman
API testing
```

You do not need a database or Python program.

The goal is to design the API interaction conceptually.

10. Thought process

Start with the action.

For GET

Ask:

```
What do I want?
```

Retrieve one existing product.

That means:

Read
↓
GET

Now identify the product.

product_id = 10

So a REST-style endpoint could be:

GET /products/10

The 10 acts as the path parameter.

If the product exists, think:

Successful read
↓
200

Then imagine a JSON response.

For POST

Ask:

What do I want?

Create a new product.

That means:

Create
↓
POST

Use:

POST /products

Now ask:

What data must the client send?

For example:

```
{  
  "name": "Laptop Stand",  
  "price": 1500  
}
```

If the server successfully creates it, think:

```
Created  
  ↓  
201
```

11. Pseudocode / API flow

No real programming code is needed.

Think about the GET request like this:

```
START  
  
client chooses GET  
  
client requests:  
  /products/10  
  
server receives product ID:  
  10  
  
server searches for product  
  
IF product exists  
  create JSON response  
  return success status  
ELSE  
  return not-found status  
  
END
```

For POST:

```
START  
  
client chooses POST  
  
client requests:  
  /products
```

```
client sends JSON product data

server checks input

IF input is valid
    create product
    return created product as JSON
    return creation success status
ELSE
    return request error

END
```

And conceptually:

```
Postman
  |
  | HTTP request
  ↓
API
  |
  | process
  ↓
HTTP response
  |
  ↓
Postman
```

12. Suggested solving approach: REST + Postman

Use this small workflow.

Part 1 — Design the REST request

Determine:

```
Operation
  ↓
HTTP method
  ↓
Resource URL
  ↓
Parameters
  ↓
JSON body if needed
```

Part 2 — Predict the response

Determine:

Success or failure?
↓
Expected status code
↓
Expected JSON

Part 3 — Imagine the Postman test

For GET:

Select GET
↓
Enter /products/10
↓
Click Send
↓
Check status
↓
Check JSON

For POST:

Select POST
↓
Enter /products
↓
Choose JSON body
↓
Enter product data
↓
Click Send
↓
Check status
↓
Check JSON response

13. Easy edge cases

Product does not exist

Request:

GET /products/999

But product 999 does not exist.

Expected concept:

404 Not Found

Missing required product data

Suppose a product requires:

```
name
price
```

but the client sends:

```
{
  "name": "Laptop Stand"
}
```

The server may reject the request because price is missing.

Beginner expectation:

400-type client error

The exact response depends on the API implementation.

Invalid JSON

The client may send malformed data that cannot be interpreted correctly.

Conceptually:

```
Malformed request
  ↓
Server cannot process input
  ↓
Client error
```

Missing API key

For a protected endpoint:

```
Request
+
No required authentication
  ↓
Authentication failure
```

A common result could be:

401 Unauthorized

14. Common mistakes to avoid

Mistake 1 — Using POST to read data

Wrong beginner mapping:

POST → read

Remember:

GET → read
POST → create

Mistake 2 — Confusing path and query parameters

Path:

/products/10

Think:

Which product?

Query:

/products?category=electronics

Think:

Which filter or option?

Mistake 3 — Confusing request and response JSON

For POST:

Client → JSON request → Server

The server may then send:

Server → JSON response → Client

They are different messages.

Mistake 4 — Checking JSON but ignoring the status code

In Postman, inspect both:

```
Status code
+
Response body
```

A JSON message alone does not tell you the entire result.

Mistake 5 — Thinking every failed request means the API is broken

Example:

```
GET /products/999
```

returning:

```
404
```

may be exactly the correct behavior if product 999 does not exist.

Testing includes checking that errors happen correctly.

Mistake 6 — Publicly exposing real API keys

Do not use:

```
real_secret_api_key
```

in learning examples or public code.

Use placeholders:

```
YOUR_API_KEY
```

15. Quick self-check questions

Try answering these without looking back.

Question 1

Which HTTP method normally retrieves data?

```
GET
POST
PUT
DELETE
```

Question 2

In:

```
/products/25
```

what does 25 represent?

Question 3

Which status code usually means a resource was successfully created?

```
200  
201  
404  
500
```

Question 4

What is the difference between:

```
/products/10
```

and:

```
/products?category=electronics
```

Question 5

After clicking **Send** in Postman, which two beginner-level things should you inspect first?

16. Hint only

For the revision problem, start with this mapping:

```
Read    → GET  
Create  → POST
```

For retrieving product 10, think:

```
/products/?
```

Ask yourself what should replace ?.

For creating a product, ask:

```
Where should the product information go?
```

Remember:

```
POST
+
JSON request body
```

For status codes, use these clues:

```
Successful read    → 2xx
Successful create → 201
Not found          → 404
Bad input          → 400-type error
```

Then imagine testing each request in Postman:

```
Method
↓
URL
↓
Body if needed
↓
Send
↓
Status code
↓
JSON response
```

Day 126 key takeaway

The entire week can be reduced to one flow:

```
Client / Postman
|
| HTTP request
| - method
| - URL
| - parameters
| - headers
| - JSON body if needed
↓
API Server
|
| process request
↓
HTTP response
| - status code
| - JSON body
↓
Client / Postman
```

And your core REST mapping remains:

GET	→ Read
POST	→ Create
PUT	→ Update
DELETE	→ Delete

You now have the API fundamentals needed to move from **conceptually understanding APIs** toward **building and testing small backend endpoints**.